סדנה מתקדמת בתכנות

פרק 3 : מבנים (רשומות)

הקדמה:

- אם צריך לשמור אוסף ערכים בעלי אותו סוג – משתמשים ב**מערך**.
- מה אם צריך לשמור נתונים שהם כולם מסוגים שונים? (למשל, אוסף נתונים עבור בן- אדם: name – מחרוזת, age – שלם וכו')
- אז מה פתרון?

טיפוס חדש : מבנה

- מבנה (Structure) הוא טיפוס נתונים מורכב בשפת C (בניגוד לטיפוס בסיסי).
- זהו טיפוס שמיועד ל**איחוד קבוצת משתנים תחת שם אחד**. משתנים אלו נקראים **שדות (איברים)** של המבנה (הרשומה), והם יכולים להיות **מטיפוסים שונים**.
- מגדירים את קבוצת המשתנים בעזרת המילה struct.
- המילה struct משמשת להגדרת הטיפוס החדש שטרם השתמשו בו. היא יכולה להופיע בכל מקום בו אפשר להגדיר טיפוס חדש.
  - כלומר, בד"כ בתחילת הקובץ.
  - או באזור הצהרת המשתנים של פונקציה כלשהי.

מבנים - תיאור פורמאלי

- הגדרת מבנה (טיפוס חדש):

```
struct structure_name
{
   type   field_name;
   . . . . .
   type   field_name;
};
```

- הצהרת משתנה מהסוג החדש שהגדרנו:

```
struct structure_name var_name;
```

- פנייה לשדה של משתנה שהוא מבנה נעשית ע"י

```
var_name.field_name
```

טיפוס חדש : מבנה

- לדוגמה נתבונן בהגדרה הבאה
(היא תהיה בקובץ tstStruct.h) :

```
struct date
{
    int day;
    int month;
    int year;
};
```

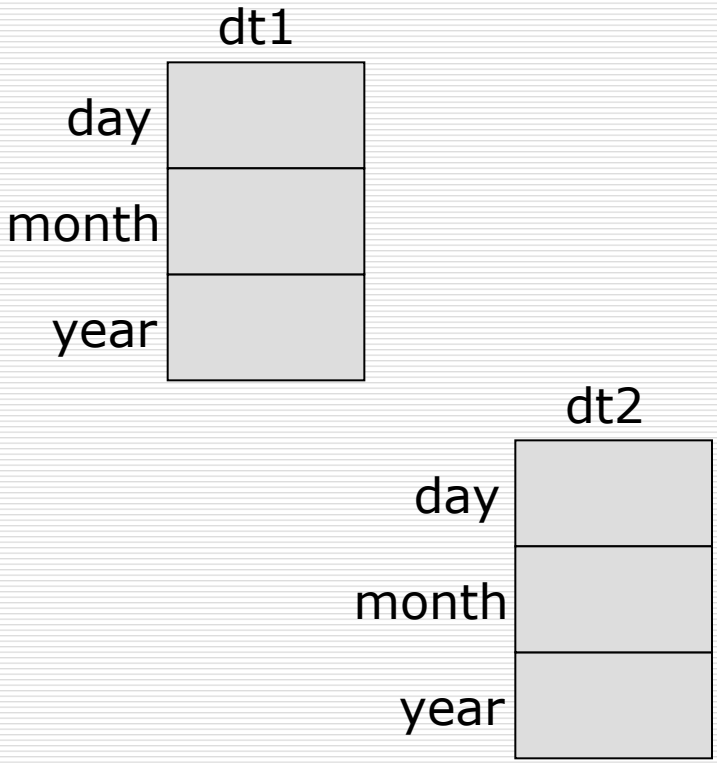
- הגדרה זו יוצרת טיפוס נתונים חדש בשם **struct date** המאחד בתוכו שלושה שדות מטיפוס int : year, month, day.

- שימו לב : הגדרה זו אינה מקצה זיכרון כלשהו, אלה רק קובעת טיפוס חדש (זו רק שבלונה ליצירת ערכים).

טיפוס חדש : מבנה

- טיפוס זה יוכל לשמש אותנו בהמשך להצהרה על משתנים.

```
#include "tstStruct.h"
void main()
{
    struct date dt1, dt2;
    ...
    dt1.day=25;
    ...
    scanf("%d", &dt2.month);
     ...
}
```



קיצורי דרך : typedef

- פקודה **typedef** יוצרת שם חדש (כינוי) לטיפוס שמוגדר קודם
- פורמט :

typedef טיפוס בסיסי שם חדש

- דוגמאות :

```
typedef char byte;
typedef struct date Date;
typedef struct date *DatePtr;
typedef int int10[10];
```

קיצורי דרך : typedef

```
typedef struct date Date;
```

- או אפשר להגדיר שם חדש ישר אחרי מבנה (בלי שם בסיסי) :

```
typedef struct {
...
} Date;
```

- או עם שם בסיסי :

```
typedef struct date {
...
} Date;
```

- דוגמה :

```
Date dt1, dt2;
```

פקודה **typedef** מאפשרת שימוש שם הטיפוס קצר יותר (בלי מילה struct)

דוגמה:

```
typedef struct person
{
    char first_name[30], last_name[30];
    unsigned id;
    int age;
} employee, student, patient;
```

שמות נוספים של הטיפוס

משתנים בעלי אותו טיפוס

```
struct person pr;
employee em;
student st;
patient pt;
```

דוגמה:

```
struct person
{
    char first_name[30], last_name[30];
    unsigned id;
    int age;
} employee, student, patient;
```

עכשיו הם שמות משתנים!

```
scanf("%d%d",&employee.age, &patient.age);
student.id = 321456789;
```

טיפוס מבנה

- אפשר באותו קובץ-h להגדיר טיפוס מבנה חדש ולהצהיר על פונקציות שמשתמשות בו

נגדיר קובץ tstStruct.h:

```
#include <stdio.h>
typedef struct date
{
    int day;
    int month;
    int year;
} Date;

Date inputDate();
void printDate(Date dt);
```

דוגמה: קלט

המימוש פעולות בקובץ tstStruct.c:

```
#include "tstStruct.h"
Date inputDate()
{
    Date dt;
    printf("Enter day(1-31): ");
    scanf("%d", &dt.day);
    printf("Enter month(1-12): ");
    scanf("%d", &dt.month);
    printf("Enter year(yy): ");
    scanf("%d", &dt.year);
    return dt;
}
```

על מנת לגשת לשדה כלשהו משמשים באופרטור נקודה (.):

dt.year ,dt.month ,dt.day

דוגמה: פלט

המימוש פעולות בקובץ tstStruct.c:

```
void printDate(Date dt)
{
    printf("( %d/ %d/ %d )", dt.day, dt.month,
    dt.year);
}
```

דוגמה: קלט\פלט

- תכנית לבדיקה:

```
#include "tstStruct.h"
void main()
{
    Date d=inputDate();
    printf("Date: ");
    printDate(d);
    printf("\n");
}
```

```
Enter day(1-31): 12
Enter month(1-12): 5
Enter year(yy): 10
Date: ( 12/ 5/ 10 )
Press any key to continue . . .
```

העתקת מבנים

- ניתן לבצע השמה של מבנה כלשהו למבנה אחר מאותו הטיפוס. פעולת השמה מעתיקה את התוכן של כל אברי המבנה לתוך המבנה השני.

```
#include "tstStruct.h"
void main()
{
    Date d1, d2;
    d1=inputDate();
    printf("\nDate source: ");
    printDate(d1);
    d2=d1;
    printf("\nDate dest: ");
    printDate(d2);
    printf("\n");
}
```

```
Enter day(1-31): 15
Enter month(1-12): 11
Enter year(yy): 85

Date source: ( 15/ 11/ 85 )
Date dest: ( 15/ 11/ 85 )
Press any key to continue . . .
```

טיפוס מבנה

- מבנה יכול להכיל משתנים מטיפוסים שונים.

לדוגמה נתבונן במבנה שמציין פרטי סטודנט תוך קובץ tstStruct.h :

```
typedef struct student
{
    unsigned id;
    char year;
    double average;
    Date birthday;
} Student;
```

- מבנה Date שהוגדר קודם הוא עצמו טיפוס לכל דבר, ולכן ניתן לכלול גם משתנה מטיפוס זה בטיפוס חדש.

אתחול מבנים

- באופן כללי מבנה – הוא רצף תאים בזיכרון (אבל מסוגים שונים) ויש אפשרות לאתחל מבנה בעזרת סוגריים מסולסלים:

```
#include "tstStruct.h"
void main()
{
    Student s={321976530, 'a', 91.6, {12,2,89}};
    . . .
}
```

אופרטור sizeof למבנים

- ניתן להפעיל את האופרטור sizeof על מנת לקבל כמות הזיכרון שצריך מבנה כלשהו.

- כמות הזיכרון שהוא צריך הוא **לא תמיד שווה** לסך הזיכרון שדורשים כל השדות שלו.
- הוא יכול להיות גם **גדול** מסך הזיכרון שדורשים כל השדות שלו.

- **למה?**

אופרטור sizeof למבנים

```
struct A
{
   char a[2]; // 2 bytes
   int b; // 4 bytes
};
```

- sizeof(a)+sizeof(b) = 2+4 = 6 bytes
- sizeof(struct A) = 8 bytes

- **למה?**

אופרטור sizeof למבנים

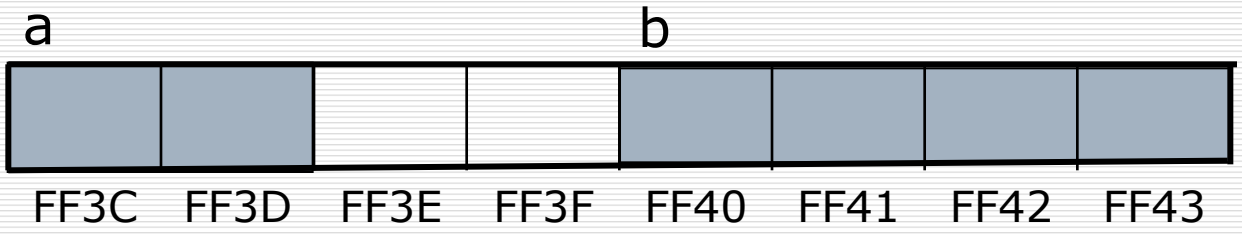
- **יישור מבנה (Structure Alignment)** מתייחס ליכולת של המהדר להוספת זיכרון שאינו בשימוש למבנה כך שאיברי המבנה יהיו מיושרים בצורה אופטימאלית עבור ביצועים טובים יותר.

- מעבדים רבים מבצעים הטוב ביותר כאשר סוגים בסיסיים של הנתונים מאוחסנים ב**כתובות** שהן **כפולות של הגדלים** שלהם.

אופרטור sizeof למבנים

```
struct A
{
    char a[2]; // 2 bytes
    int b; // 4 bytes
};

struct A  x;
```



אופרטור sizeof למבנים

```
struct X  /* 12 bytes */
{
    short s;  /* 2 bytes */
              /* 2 padding bytes */
    int     i;  /* 4 bytes */
    char  c;  /* 1 byte */
              /* 3 padding bytes */
};

struct Y  /* 8 bytes */
{
    int     i;  /* 4 bytes */
    char  c;  /* 1 byte */
              /* 1 padding byte */
    short s;  /* 2 bytes */
};
```

אופרטור sizeof למבנים

```
struct Z /* 8 bytes */
{
    int     i;  /* 4 bytes */
    short s;  /* 2 bytes */
    char  c;  /* 1 byte */
              /* 1 padding byte */
};

struct W /* 8 bytes */
{
    char  c;  /* 1 byte */
              /* 1 padding byte */
    short s;  /* 2 bytes */
    int     i;  /* 4 bytes */
};
```

אופרטור sizeof למבנים

- אפשר למזער את הגודל של מבנים על ידי הצבת טיפוסי הנתונים הגדולים ביותר בתחילתו או בסופו של המבנה.
- כדאי להצהיר איברי המבנה בסדר עולה או בסדר יורד של גודלם.
- כל מהדר יכול לבחור ליישר נתונים באופן שונה, מה שכתוצאה מוביל למפריסות נתונים שונות ולא תואמות.

אופרטור sizeof למבנים

```
struct A1
{
    char a;
    double b;
    char c;
};

printf ("struct A1 = %d bytes\n", sizeof(struct A1));
```

```
struct A1 = 24 bytes
```

אופרטור sizeof למבנים

```
struct A2
{
    double b;
    char a;
    char c;
};

printf ("struct A2 = %d bytes\n", sizeof(struct A2));
```

```
struct A2 = 16 bytes
```

אופרטור sizeof למבנים

```
struct A3
{
    char a;
    char c;
    double b;
};

printf ("struct A3 = %d bytes\n", sizeof(struct A3));
```

```
struct A3 = 16 bytes
```

אופרטור sizeof למבנים

- הפעלת האופרטור sizeof למבנה Student:

```
...
printf("sizeof Student: %d\n", sizeof (Student));
printf("sizeof s.id: %d\n", sizeof s.id);
printf("sizeof s.year: %d\n", sizeof s.year);
printf("sizeof s.average: %d\n", sizeof s.average);
printf("sizeof s.birthday: %d\n", sizeof s.birthday);
...
```

אופרנד של **sizeof** שהוא אובייקט
(לא טיפוס) אפשר לכתוב בלי סוגריים

אופרטור sizeof למבנים

פלט:

```
sizeof Student: 32
sizeof s.id: 4
sizeof s.year: 1
sizeof s.average: 8
sizeof s.birthday: 12
Press any key to continue . . .
```

$4+1+8+12=25\leq 32$

מערך של מבנים

- ב- C ניתן ליצור מערך מכל טיפוס!! גם ממבנים.

```
#include "tstStruct.h"
void main()
{
    Date borns[3]={{12,7,80}, {10,3,85}, {5, 5, 90}};
    int i;
    for(i=0; i<3; i++)
    {
        printDate(borns[i]);
        printf("\n");
    }
}
```

```
( 12/ 7/ 80 )
( 10/ 3/ 85 )
( 5/ 5/ 90 )
Press any key to continue . . .
```

גודלו המערך :

גודלו המערך borns :

3*sizeof(Date) = 3*12 = 36

- שימוש בשדה אחד או יותר :

```
...
    borns[1].day=13;
    printDate(borns[1]);
...
```

```
( 13/ 3/ 85 )
Press any key to continue . . .
```

מבנים ומצביעים

```
#include "tstStruct.h"

void main()
{
    Student s={345678901, 'b', 91.2, {12,2,89}};
    Student *sptr=&s; //pointer to struct
    printf("\nID: %u\n", sptr->id); //(*sptr).id or s.id
    printDate(sptr->birthday); //s.birthday
```

```
ID: 345678901
( 12/ 2/ 89 )
The year of birth is 92
Press any key to continue . . .
```

שינוי שדה מספרי:

```
    sptr->birthday.year=92; //s.birthday.year
    printf("\nThe year of birth is %d\n", sptr->birthday.year);
}
```

מבנים ומצביעים

- בעזרת המצביע sptr נוכל לגשת לשדות פנימיים של Student בשתי דרכים:

```
(*sptr).id
(*sptr).birthday
…
```

⟺

```
sptr -> id
sptr -> birthday
…
```

- משמעות האופרטור -> הוא לגשת לשדה תוך מבנה שמצביע sptr מצביע עליו.

מערך דינאמי של מבנים

```
#include "tstStruct.h"
void main()
{
    Date *borns;
    int i, n;
    scanf("%d",&n);
    borns=(Date *)calloc(n,sizeof(Date));
    for(i=0; i<n; i++)
        borns[i]=inputDate();
    . . .
}
```

פעולות על מבנים

- השמה של מבנה למבנה אחר בעל אותו טיפוס =
- לקיחת כתובת של מבנה &
- גישה לאיברי מבנה .
- שימוש באופרטור sizeof להגדרת גודל מבנה

- אין פעולות השוואה למבנים!! אפשר להשוות שדות של מבנים מאותו סוג אבל אי-אפשר להשוות מבנים עצמם! השוואת מבנים אסורה כדי להימנע מהשוואת תכני המרווחים בין איברי המבנה.

מבנה כפרמטר של פונקציה

- ניתן להעביר כל מבנה כפרמטר לפונקציה. כברירת מחדל העברה מתבצעת by-value (כלומר כל השדות של המבנה מועתקים לפרמטרים פורמאליים של פונקציה).
  העתקה מתבצעת כולל **מערכים**.
  לכן כל שינוי שנתבצע תוך פונקציה לא ישפיע על מבנה המקורי!!

- לדוגמה: פונקציה הבאה מחשבת ומדפיסה יום הבא:

דוגמה:

```
void printNextDay(Date d)
{
    d.day++;
    if(d.day>31)
    {
        d.day=1; d.month++;
        if(d.month>12)
        {
                d.month=1;
                d.year++;
        }
    }
    printf("\n Next date:");
    printDate(d);
    printf("\n");
}
```

ריצה:

- נריץ תת-המשימה:

```
#include "tstStruct.h"
void main()
{
    Date dt={31,12,2009};
    printf("source date:");
    printDate(dt);
    printNextDay(dt);
    printf("source date:");
    printDate(dt);
    printf("\n");
}
```

```
source date:( 31/ 12/ 2009 )
 Next date:( 1/ 1/ 2010 )
source date:( 31/ 12/ 2009 )
Press any key to continue . . .
```

By-reference

כמובן הביצוע שונה אם מעבירים מבנה בשיטת by-reference:

```
void printNextDayP(Date *d)
{
    d->day++;
    if(d->day>31)
    {
        d->day=1; d->month++;
        if(d->month>12)
        {
                d->month=1;
                d->year++;
        }
    }
    printf("\n Next date:");
    printDate(*d);
    printf("\n");
}
```

ריצה:

```
#include "tstStruct.h"
void main()
{
    Date dt={31,12,2009};
    printf("source date:");
    printDate(dt);
    printNextDayP(&dt);
    printf("source date:");
    printDate(dt);
    printf("\n");
}
```

```
source date:( 31/ 12/ 2009 )
 Next date:( 1/ 1/ 2010 )
source date:( 1/ 1/ 2010 )
Press any key to continue
```

העברת מערך by value דרך struct

```
typedef struct
{
    int arr[100];
}   int_array_100;

void func_name(int_array_100 a)
{   ...  }

int_array_100 A;

func_name(A); // function call
```

פניה לאיבר i בתוך המערך: a.arr[i]

מצביע כשדה של מבנה

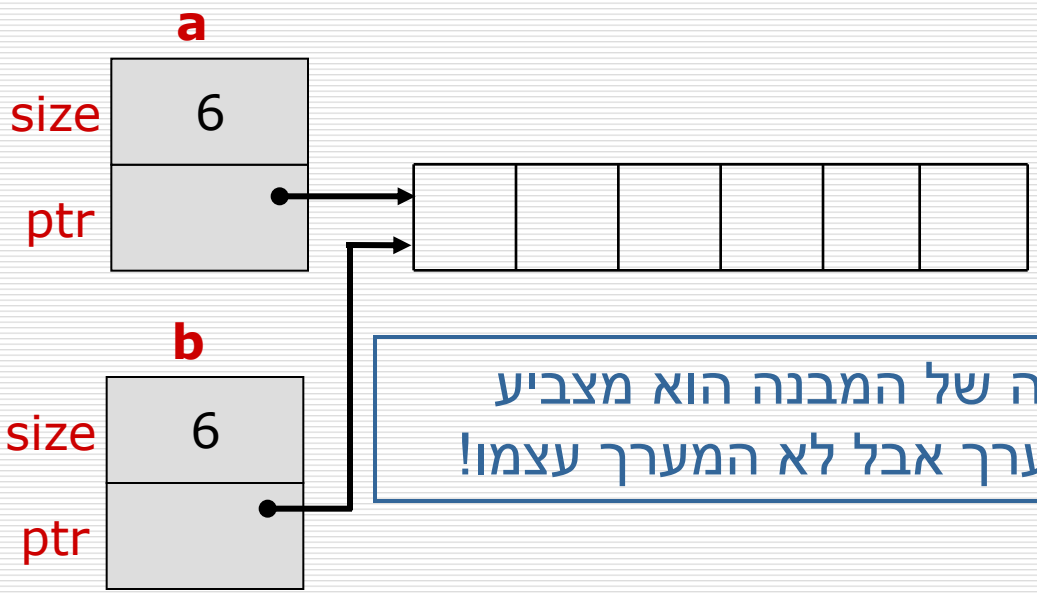
```
typedef struct
{
    int size;
    int *ptr;
}   dyn_array;

dyn_array a;
int i;
scanf("%d",&a.size);
a.ptr=(int *)calloc(a.size, sizeof(int));
for(i=0; i<a.size; i++)
    scanf("%d",a.ptr+i);
```

מצביע כשדה של מבנה

```
typedef struct
{
    int size;
    int *ptr;
}   dyn_array;

dyn_array a, b;
int i;
scanf("%d",&a.size);
a.ptr=(int *)calloc(a.size, sizeof(int));
.....
b=a;
```



שדה של המבנה הוא מצביע
למערך אבל לא המערך עצמו!

מעתיקים רק מצביע למערך אבל לא את המערך עצמו!
לכן a.ptr[i] שקול ל- b.ptr[i]